

CLIENT DELIVERY DOCUMENT

Hierarchia Manager Portal User Guide

Version: 1.0

Date: May 12, 2026

Prepared by: Shahroz Imran

Classification: Client Confidential

Client Delivery Document | Version 1.0 | May 12, 2026

Prepared by: Shahroz Imran

1. Purpose of This Document

This document explains, in client-friendly terms, **why the project became more complex during implementation** and why several major architectural decisions were necessary.

From the outside, some of these changes may appear to be simple feature additions or hosting changes. In reality, many of them required **deep restructuring of the platform**, because the system evolved from a standard portal into a much more advanced product that includes:

- AI-powered document validation
- Smart AI with RAG-based document understanding
- Real-time messaging and collaboration
- AI access control and usage governance
- Self-hosted infrastructure and security hardening

The purpose of this note is to clarify that these were not minor adjustments. They were **platform-level engineering changes** that increased both technical depth and delivery complexity.

2. Why the Project Complexity Increased Over Time

The project did not remain a fixed-scope dashboard application.

Instead, it evolved in stages:

1. A role-based manager portal was built.
2. AI validation workflows were added.
3. The Smart AI assistant was expanded with RAG and live data awareness.
4. The architecture was shifted away from the earlier Vercel-centered deployment model to a Railway-based architecture.
5. Messaging and internal collaboration were integrated.
6. AI access control, permissions, and usage limits were introduced.

Each of these stages affected not only the visible interface, but also the:

- backend architecture
- database design

- security model
- deployment setup
- background processing model
- system reliability and recovery logic

This is the main reason the implementation became significantly more involved than a typical admin portal project.

3. Major Architectural Changes That Increased Complexity

3.1 Migration from Vercel-Oriented Architecture to Railway-Based Architecture

One of the biggest changes was the move from an earlier **Vercel-oriented structure** to a more complete **Railway-hosted architecture**.

This was not just a hosting switch.

It required reworking how the application handled:

- backend deployment
- environment configuration
- authentication flow
- scheduled jobs
- Redis-based rate limiting
- database connectivity
- service-to-service communication
- recovery logic for failed or stuck AI operations

In practical terms, this meant the system had to be adapted to run reliably in a new runtime environment, with different operational behavior and infrastructure assumptions.

Why this was complex

- Vercel-style assumptions could not simply be copied into Railway.
- Session handling, background execution, and infrastructure connectivity had to be revalidated.
- Cron and recovery behavior had to be redesigned for the new deployment model.
- The platform had to remain stable while core architecture was changing underneath it.

This type of migration impacts the whole system, not just one module.

3.2 RAG and Smart AI Architecture Shift

Another major source of complexity was the evolution of the **Smart AI assistant**.

Initially, the AI layer was simpler. Over time, it had to support:

- document-aware responses
- retrieval of relevant indexed content
- live platform data access
- persistent chat threads
- better reliability and control over AI behavior

To achieve this, the project moved toward a **native Smart AI architecture** where retrieval, orchestration, and business-aware logic are handled directly inside the application architecture.

This also included the transition away from a more separated MCP-style/external orchestration pattern into a tighter, platform-native implementation better suited to the delivered system.

Why this was complex

RAG is not just a chatbot feature. It requires:

- document ingestion
- text extraction
- chunking strategy
- embeddings generation
- searchable indexing
- permission-aware retrieval
- controlled AI prompting
- error handling when retrieval or indexing fails

On top of this, the Smart AI assistant was expected to work with **real business data**, not just static text. That meant the AI layer had to be integrated carefully with system permissions, role boundaries, and data visibility rules.

In simple terms, the AI needed to be **useful, accurate, secure, and scoped correctly** for each user. That is a much harder problem than adding a generic AI chat box.

3.3 Full Messaging Integration Inside the Portal

The integration of **real-time messaging** added another major layer of complexity.

This was important because the portal was no longer only a task and submission system. It became a collaboration platform where users could communicate directly inside the product.

The messaging module included:

- direct messages
- group conversations
- real-time delivery
- read/unread state tracking
- typing indicators
- reactions and replies
- edit/delete flows
- file and media attachments

Why this was complex

Messaging systems look simple on the front end, but are technically demanding because they require:

- persistent conversation structure
- real-time synchronization across users
- careful permission enforcement
- attachment handling and storage
- performance optimization for active conversations
- data consistency for edits, deletes, and reactions
- safe handling of user presence and membership rules

This changed the project from a workflow portal into a more advanced, multi-user communication environment.

3.4 AI Access Control and Usage Governance

Once Smart AI became a meaningful feature, it was not enough to simply make it available to everyone without limits.

The platform needed **AI access control**, which introduced another serious architecture layer.

This included:

- role-aware AI permissions
- per-user access enable/disable control
- AI credit allocation and limits
- usage tracking
- usage history
- near-limit alerts

- admin-level control over AI consumption

Why this was complex

AI access control combines business rules, security rules, and cost-control rules.

That means the system had to decide:

- who can use AI
- which AI capabilities they can use
- what documents or data they are allowed to query
- how much usage is allowed per user or period
- how administrators can monitor and manage usage

This is especially important in enterprise-style systems, because unrestricted AI access can create:

- security exposure
- role violations
- uncontrolled AI cost growth
- inconsistent user experience

So this work was not only about adding limits. It was about building a **governed AI layer** suitable for real organizational use.

4. Hidden Engineering Complexity Behind the Delivered Features

The table below summarizes why some seemingly simple features required large technical effort.

Visible Outcome	Hidden Engineering Work Required
Move from Vercel to Railway	Reworking deployment architecture, environment handling, auth/session behavior, cron strategy, Redis integration, and operational recovery
Smart AI with RAG	Building document ingestion, indexing, retrieval, permission-aware search, AI orchestration, and fallback/error handling
Messaging inside the portal	Designing conversation models, realtime sync, attachment pipelines, reaction/edit/delete logic, unread tracking, and performance tuning
AI access control	Building permissions, quotas, credit tracking, admin controls, logs, alerts, and role-aware AI boundaries
Stronger architecture overall	Updating database design, APIs, background jobs, security layers, and reliability behavior across the platform

5. Risks That Had To Be Managed During These Changes

Because the architecture changed while the product scope was also expanding, the implementation had to carefully manage several risks:

- **Continuity risk**
 - Existing modules still needed to keep working while infrastructure and architecture changed.
- **Security risk**
 - AI, messaging, and document retrieval all introduced new access-control concerns.
- **Data visibility risk**
 - Users should only see the messages, documents, and AI outputs they are authorised to access.
- **Reliability risk**
 - Background AI processing, uploads, and retrieval had to recover safely if anything became stuck or failed.
- **Performance risk**
 - Real-time messaging and AI retrieval both place heavier demands on the system than a standard dashboard.
- **Operational risk**
 - Self-hosted and multi-service architecture requires more careful setup, monitoring, and deployment discipline.

These risks were not theoretical. They directly influenced the engineering decisions required to complete the platform properly.

6. Why These Changes Were Necessary

These changes were necessary because the final delivered product is not a basic portal.

It is a more advanced operational platform that combines:

- workflow management
- secure file handling
- AI-powered validation
- AI-assisted retrieval and analysis
- real-time collaboration
- administrative governance

Without these architectural changes, the system would have faced limitations in:

- scalability
- maintainability
- security
- reliability
- feature growth

In other words, the added complexity was not unnecessary overhead. It was the engineering work required to make the platform capable of supporting the final business goals.

7. Client-Facing Summary

The main point for the client is this:

The project complexity increased because the platform evolved from a standard management portal into a secure, AI-enabled, real-time collaboration system with self-hosted infrastructure and governed AI access.

What may appear as a small set of visible modules on the screen actually required:

- architecture redesign
- infrastructure migration
- database expansion
- security hardening
- AI system integration
- realtime communication engineering
- usage governance and recovery mechanisms

This is why the implementation effort and technical depth became significantly greater than in a conventional dashboard or CRUD-based portal project.

8. Closing Note

The delivered system reflects not only feature development, but also **major architectural evolution** carried out during implementation.

This work was essential to ensure that the final product is:

- functionally rich
- secure

- scalable
- maintainable
- suitable for real client use in production

For that reason, the complexity encountered during development should be understood as a direct result of building a more capable and production-ready platform than the original baseline architecture.